Herald

Protocol Research: NATS JetStream

Field	Value
Revision	1
Created	2026-05-22
Last modified	2026-05-22
Status	research-only (no implementation yet)
Status summary	NATS is a high-performance, cloud-native pub/sub system. JetStream (NATS 2.2+) adds persistence + at-least-once / exactly-once-within-window delivery, making it a viable alternative to Kafka for event ingestion. For Herald, NATS is positioned as a P4 ALTERNATIVE to Kafka — pick one, not both, for MVP. NATS wins for low-latency / Kubernetes-native scenarios; Kafka wins for big-data / replay-heavy.
Issues	open-questions: pick Kafka or NATS for MVP; in-process embedded server for tests; subject naming.
Continuation	Wave 4f+ alternative — open HRD block only if operator picks NATS instead of (or in addition to) Kafka.

Constitutional anchors

- **§107 anti-bluff** — tests boot a real `nats-server` with JetStream enabled via `containers/`; real `nats pub` CLI publishes; Herald consumes; assert delivery + dedupe.
- **§11.4.74 catalogue-check** — no `vasic-digital` or `HelixDevelopment` NATS module. Use `github.com/nats-io/nats.go` (official; Synadia / NATS team maintained).
- **§11.4.61** — tracked doc.

Table of contents

§1. Protocol overview

NATS. Originally a lightweight pub/sub (Apcera, 2014). Now part of CNCF. Apache 2.0. Two API tiers:

- **Core NATS** — best-effort pub/sub. At-most-once. Microsecond latency. No persistence.
- **JetStream** (NATS 2.2+) — persistence layer on top of Core. At-least-once + exactly-once-within-window. KV store + object store as secondary APIs.

Wire format. Plain text protocol (line-delimited; verbs: `PUB`, `SUB`, `MSG`, `INFO`, ...) plus binary headers. Default TCP port 4222.

Topology. Pub/sub via subjects (`foo.bar.baz`). Wildcards: `*` (token), `>` (rest).

Authentication. Plain token, NKeys (Ed25519), JWT, mTLS.

Adoption. Used by big cloud-native shops — Pleo, Choria, Synadia clients. Heavy in service-mesh / Kubernetes scenarios.

§2. Specification deep-dive

§2.1 Core NATS

- `SUB <subject> <sid>` — subscribe.
- `PUB <subject> [reply-to] <length>\r\n<payload>\r\n` — publish.
- `MSG <subject> <sid> [reply-to] <length>\r\n<payload>\r\n` — incoming.
- `PING` / `PONG` — keepalive.

§2.2 JetStream — streams + consumers

- **Stream** — append-only sequence of messages with retention policy.
- **Consumer** — view over a stream; pull-based or push-based.

Consumer types:

- **Push** — server delivers to client's subscription.
- **Pull** — client requests N messages at a time.

Delivery semantics:

- **At-least-once** — default; manual ack.
- **Exactly-once-within-window** — `Msg-Id` header + dedup window; producer-set message-id; broker dedupes.

§2.3 Message headers (NATS 2.2+)

Similar to HTTP headers. CloudEvents NATS binding uses these.

§2.4 KV store + object store

JetStream's higher-level APIs:

- **KV** — get/put/watch on key namespaces.
- **Object** — large blob storage (chunked).

Useful for: caches, leader election, config distribution.

§3. Herald-specific applicability analysis

§3.1 Use case

Same as Kafka: Herald INGESTS NATS subjects as events. Example:

- Microservices publish to `audit.*`.
- Herald subscribes to `audit.>`.

§3.2 NATS vs Kafka — pick one

Aspect	NATS	Kafka
Latency	µs	low-ms
Throughput	high	very-high
Replay window	configurable (smaller)	configurable (longer)
Resource footprint	small	larger
K8s-native	yes (CNCF)	yes
Big-data analytics	no	yes (KSQL etc.)
Cloud managed	Synadia	many (Confluent, MSK, Event Hubs)

For Herald (event fan-out, not analytics): NATS is plenty. **Recommendation: NATS for MVP if anything; Kafka only if operator has existing Kafka commitment.**

§3.3 Herald is a CLIENT

Use external NATS server. Don't run NATS inside Herald.

(Exception: in-process embedded NATS server for TESTS is acceptable — `nats-server` can be embedded as a Go library for hermetic test runs.)

§3.4 Multi-tenant

NATS subject hierarchy maps cleanly: `tenant.<tenant_id>.events.>` per tenant. Subscribe with wildcard per tenant.

Or: per-tenant connection with per-tenant credentials.

§3.5 Idempotency

`Msg-Id` header + JetStream dedup window → broker-side dedup. Belt-and-suspenders with Herald's Redis SETNX.

§3.6 CloudEvents NATS binding

Headers prefixed `ce-` (note: dash, like HTTP). Body = CloudEvent `data`.

§3.7 OTel propagation

`ce-traceparent` header → Herald extracts.

§3.8 Failure modes

1. **Server down.** Auto-reconnect with jitter; nats.go has this.
2. **Consumer ack timeout.** If Herald doesn't ack within ack_wait, message redelivered. Tune ack_wait.
3. **Stream storage full.** Monitor; alert; consider auto-purge.

§4. Step-by-step implementation guide for Herald

§4.1 Add dep

Submodule: `github.com/nats-io/nats.go` under `submodules/go-nats/`.

§4.2 New channel adapter

`commons_messaging/channels/nats/`.

§4.3 Subscriber config

- `nats_url` — `nats://broker.example.com:4222`.
- `nats_subject` — wildcard pattern.
- `nats_consumer_name` (for JetStream).
- `nats_auth_token` OR `nats_jwt`.

§4.4 In-process server for tests

```
import "github.com/nats-io/nats-server/v2/server"

opts := &server.Options{JetStream: true, Port: -1 /* random */}
ns, _ := server.NewServer(opts)
go ns.Start()
defer ns.Shutdown()
```

Hermetic; no `containers/` boot needed for fast unit tests.

§4.5 New e2e_bluff_hunt invariants

- **E108** — boot nats-server with JetStream; `nats pub foo.bar 'hello'`; Herald consumes; event in DB.
- **E109** — JetStream durability: kill broker; restart with same storage; events persisted; Herald resumes.
- **E110** — exactly-once-within-window: produce duplicate `Msg-Id`; assert single delivery.
- **E111** — CloudEvents NATS binding: ce- headers propagate.

- **E112** — KV: write/read.

§4.6 HRD scaffolding (if NATS chosen)

- HRD-3xx — vendor nats.go.
- HRD-3xx — channel adapter.
- HRD-3xx — JetStream stream + consumer mgmt.
- HRD-3xx — CloudEvents binding.
- HRD-3xx — egress publisher.
- HRD-3xx — auth (NKey / JWT).
- HRD-3xx — e2e_bluff_hunt E108–E112.
- HRD-3xx — close Wave 4f (NATS slice).

§5. §107 anti-bluff testing strategy

§5.1 Happy-path

- **Test 1: pub/consume.** real `nats pub` from CLI → Herald consumes → events_processed.
- **Test 2: JetStream stream.** publish to a stream; Herald pull-subscribes; assert receive + ack.
- **Test 3: CloudEvents binding.** ce-id header preserved.

§5.2 Reliability

- **Test 4: redelivery.** Don't ack; ack_wait expires; message redelivered.
- **Test 5: dedup window.** Same Msg-Id twice within window → single delivery.
- **Test 6: durable consumer.** Crash Herald; restart; resume from last ack.

§5.3 Failure modes

- **Test 7: server down.** Reconnect.
- **Test 8: auth fail.** Token rejected; backoff.

§5.4 Performance

- **Test 9: throughput.** 100K msgs/sec sustained (NATS shines here).

§5.5 Mutation gates

- Mutation: skip ack → Test 4 sees redelivery storm.
- Mutation: skip Msg-Id dedup → Test 5 FAILS.

§5.6 Wire-level

- tcpdump :4222 → readable text protocol; verify PUB / MSG sequence.

§6. Open questions for operator

1. **Kafka or NATS for Herald MVP?** Recommend NATS if operator has no Kafka pre-existing — simpler ops; lighter.

2. **JetStream or Core NATS?** Recommend JetStream — Core has at-most-once semantics that conflict with Herald's reliability needs.
3. **Embedded test server?** Yes; faster than containers/-boot.
4. **NKey / JWT auth or plain token?** Recommend JWT (matches Herald's commons_auth pattern).

§7. References

(All fetched 2026-05-22.)

- **NATS docs.** https://docs.nats.io/
- **JetStream docs.** https://docs.nats.io/nats-concepts/jetstream
- **nats.go.** https://github.com/nats-io/nats.go
- **nats-server.** https://github.com/nats-io/nats-server
- **CloudEvents NATS binding.** https://github.com/cloudevents/spec/blob/v1.0.2/cloudevents/bindings/nats-protocol-binding.md
- **2026 implementation guide.** https://shijuvar.medium.com/building-distributed-event-streaming-systems-in-go-with-nats-jetstream-3938e6dc7a13
- **Watermill NATS pubsub.** https://watermill.io/pubsubs/nats/

§8. Catalogue-check verdict

- **vasic-digital:** no module.
- **HelixDevelopment:** no module.

Verdict: no-match → vendor `github.com/nats-io/nats.go` **(and optionally** `github.com/nats-io/nats-server/v2` **for in-process test broker) as submodules.**